

DOCUMENTACIÓN TÉCNICA - HUELLITASOFT BACKEND

INFORMACIÓN GENERAL DEL PROYECTO

Descripción del Sistema

Huellitasoft es un sistema integral de gestión veterinaria desarrollado como una aplicación backend RESTful utilizando Spring Boot. El sistema proporciona una solución completa para la administración de clínicas y consultorios veterinarios, permitiendo la gestión de usuarios, pacientes (mascotas), citas, consultas médicas, tratamientos, historiales clínicos y esquemas de vacunación.

Información del Repositorio

- **Nombre del Proyecto:** Backend_Huellitasoft
- **Propietario:** W1llAn
- **Rama Principal:** develop
- **Versión:** 0.0.1-SNAPSHOT
- **Grupo:** huellitasoft-web
- **Artefacto:** huellitasoft

Stack Tecnológico

Framework y Lenguaje

- **Java:** Versión 21 (LTS)
- **Spring Boot:** 3.5.6
- **Maven:** Sistema de gestión de dependencias y construcción del proyecto

Dependencias Principales de Spring

- **Spring Boot Starter Data JPA:** Persistencia de datos y ORM
- **Spring Boot Starter Security:** Autenticación y autorización
- **Spring Boot Starter Validation:** Validación de datos de entrada
- **Spring Boot Starter Web:** Desarrollo de API REST
- **Spring Boot Starter OAuth2 Resource Server:** Validación de tokens JWT

- **Spring Boot Starter Mail:** Envío de correos electrónicos
- **Spring Boot Starter WebSocket:** Comunicación en tiempo real
- **Spring Boot Starter Thymeleaf:** Motor de plantillas para correos

Base de Datos

- **PostgreSQL:** Base de datos relacional principal
- **Hibernate:** ORM (Object-Relational Mapping)
- **Dialecto:** PostgreSQLDialect
- **Estrategia DDL:** validate (validación sin modificación automática del esquema)

Seguridad y Autenticación

- **Auth0:** Proveedor de identidad OAuth2
- **JWT (JSON Web Tokens):** Tokens de autenticación stateless
- **BCrypt:** Algoritmo de encriptación para contraseñas

Documentación de API

- **Springdoc OpenAPI:** 2.8.13
- **Swagger UI:** Interfaz interactiva para la documentación de la API
- **Ruta de acceso:** /swagger-ui.html y /v3/api-docs

Herramientas de Desarrollo

- **Lombok:** 1.18.34 - Reducción de código boilerplate
- **Spring Dotenv:** 4.0.0 - Gestión de variables de entorno
- **SLF4J:** Framework de logging

Herramientas de Testing

- **Spring Boot Starter Test:** Framework de pruebas
- **Spring Security Test:** Pruebas de seguridad

ARQUITECTURA DEL SISTEMA

Patrón Arquitectónico

El proyecto implementa una arquitectura en capas (Layered Architecture) con separación clara de responsabilidades:

CAPA DE PRESENTACIÓN

(Controllers)

- Gestiona peticiones HTTP
- Valida datos de entrada
- Retorna respuestas HTTP



CAPA DE APLICACIÓN

(DTOs/Mappers)

- Transforma datos entre capas
- Define contratos de comunicación



CAPA DE NEGOCIO

(Services)

- Implementa lógica de negocio
- Gestiona transacciones
- Aplica reglas de negocio



CAPA DE PERSISTENCIA

(Repositories)

- Acceso a datos
- Queries personalizadas
- Gestión de entidades JPA



CAPA DE DATOS

(Entities)

- Modelos de dominio
- Mapeo objeto-relacional
- Relaciones entre entidades

Estructura de Paquetes

```
huellitasoft_web.huellitasoft/
|
├─ config/                                # Configuraciones del sistema
|   ├── AudienceValidator.java            # Validación de audiencia JWT
|   ├── CorsConfig.java                  # Configuración CORS
|   ├── CustomJwtAuthenticationConverter.java # Conversión JWT
|   ├── MailConfig.java                  # Configuración de correo
|   ├── SecurityConfig.java              # Configuración de seguridad
|   ├── SwaggerConfig.java               # Configuración de Swagger
|   └── WebSocketConfig.java             # Configuración WebSocket
|
├─ controller/                            # Controladores REST
|   ├── AppointmentController.java        # Gestión de citas
|   ├── ClientController.java             # Gestión de clientes
|   ├── ConsultationController.java       # Gestión de consultas
|   ├── MedicalHistoryController.java     # Historiales clínicos
|   ├── NotificationController.java       # Notificaciones
|   ├── PetController.java                # Gestión de mascotas
|   ├── PetSchemeController.java          # Esquemas de mascotas
|   ├── PetVaccinationController.java     # Vacunación de mascotas
|   ├── RaceController.java              # Gestión de razas
|   ├── SpecieController.java             # Gestión de especies
|   ├── SubsidiaryController.java         # Gestión de sucursales
|   ├── TreatmentController.java          # Gestión de tratamientos
|   ├── UserController.java               # Gestión de usuarios
|   ├── VaccinationSchemeController.java  # Esquemas de vacunación
|   └── VaccineController.java            # Gestión de vacunas
|
├─ dto/                                   # Data Transfer Objects
|   ├── appointment/                     # DTOs de citas
|   ├── client/                          # DTOs de clientes
|   ├── consultation/                    # DTOs de consultas
|   ├── medicalhistory/                  # DTOs de historiales
|   ├── Notification/                    # DTOs de notificaciones
|   ├── pet/                             # DTOs de mascotas
|   ├── petScheme/                       # DTOs de esquemas de mascotas
|   ├── petVaccination/                  # DTOs de vacunación
|   ├── race/                            # DTOs de razas
|   ├── Schedule/                        # DTOs de horarios
|   ├── specie/                          # DTOs de especies
|   └── Subsidiary/                      # DTOs de sucursales
```

- | └─ treatment/ # DTOs de tratamientos
- | └─ user/ # DTOs de usuarios
- | └─ vaccinationScheme/ # DTOs de esquemas de vacunación
- | └─ vaccine/ # DTOs de vacunas
- |
- |─ entity/ # Entidades JPA
 - | └─ Appointment.java # Entidad de citas
 - | └─ Client.java # Entidad de clientes
 - | └─ Consultation.java # Entidad de consultas
 - | └─ MedicalHistory.java # Entidad de historiales
 - | └─ Notification.java # Entidad de notificaciones
 - | └─ Pet.java # Entidad de mascotas
 - | └─ PetScheme.java # Entidad de esquemas de mascotas
 - | └─ PetVaccination.java # Entidad de vacunación
 - | └─ Race.java # Entidad de razas
 - | └─ Specie.java # Entidad de especies
 - | └─ Subsidiary.java # Entidad de sucursales
 - | └─ SubsidiarySchedule.java # Entidad de horarios
 - | └─ Treatment.java # Entidad de tratamientos
 - | └─ User.java # Entidad de usuarios
 - | └─ VaccinationScheme.java # Entidad de esquemas de vacunación
 - | └─ Vaccine.java # Entidad de vacunas
- |
- |─ enums/ # Enumeraciones
 - | └─ ClientState.java # Estados de cliente
 - | └─ DayOfWeek.java # Días de la semana
 - | └─ EstadoCita.java # Estados de cita
 - | └─ MedicalHistoryState.java # Estados de historial
 - | └─ NotificationTitle.java # Títulos de notificación
 - | └─ PetSchemeState.java # Estados de esquema de mascota
 - | └─ Sex.java # Sexo de mascota
 - | └─ ShiftType.java # Tipos de turno
 - | └─ SubsidiaryState.java # Estados de sucursal
 - | └─ UserRol.java # Roles de usuario
 - | └─ UserState.java # Estados de usuario
- |
- |─ exception/ # Gestión de excepciones
 - | └─ ErrorResponse.java # Estructura de respuesta de error
 - | └─ GlobalExceptionHandler.java # Manejador global de excepciones
 - | └─ ResourceAlreadyExistsException.java # Excepción de recurso existente
 - | └─ ResourceNotFoundException.java # Excepción de recurso no encontrado
- |
- |─ repository/ # Repositorios JPA

```
| |─ AppointmentRepository.java
| |─ ClientRepository.java
| |─ ConsultationRepository.java
| |─ MedicalHistoryRepository.java
| |─ NotificationRepository.java
| |─ PetRepository.java
| |─ PetSchemeRepository.java
| |─ PetVaccinationRepository.java
| |─ RaceRepository.java
| |─ SpecieRepository.java
| |─ SubsidiaryRepository.java
| |─ SubsidiaryScheduleRepository.java
| |─ TreatmentRepository.java
| |─ UserRepository.java
| |─ VaccinationSchemeRepository.java
| |─ VaccineRepository.java
|
|─ service/                                # Interfaces de servicios
| |─ AppointmentService.java
| |─ ClientService.java
| |─ ConsultationService.java
| |─ MedicalHistoryService.java
| |─ PetSchemeService.java
| |─ PetService.java
| |─ PetVaccinationService.java
| |─ RaceService.java
| |─ SpecieService.java
| |─ SubsidiaryService.java
| |─ TreatmentService.java
| |─ UserService.java
| |─ VaccinationSchemeService.java
| |─ VaccineService.java
|
|─ service/impl/                          # Implementaciones de servicios
| |─ AppointmentServiceImpl.java
| |─ ClientServiceImpl.java
| |─ ConsultationServiceImpl.java
| |─ EmailService.java
| |─ MedicalHistoryServiceImpl.java
| |─ NotificationService.java
| |─ PetSchemeServiceImpl.java
| |─ PetServiceImpl.java
| |─ PetVaccinationServiceImpl.java
```

```
├─ RaceServiceImpl.java
├─ SpecieServiceImpl.java
├─ SubsidiaryServiceImpl.java
├─ TreatmentServiceImpl.java
├─ UserServiceImpl.java
├─ VaccinationSchemeServiceImpl.java
├─ VaccineServiceImpl.java
└─ WebSocketService.java
```

MODELO DE DATOS

Diagrama de Relaciones Entre Entidades

El sistema implementa las siguientes entidades principales con sus relaciones:

Entidades Core del Sistema

1. User (Usuario)

- Representa los usuarios del sistema
- Roles: VETERINARIO, ADMINISTRADOR, CLIENTE, ADMINISTRADOR_VETERINARIA
- Estados: Activo/Inactivo
- Relación recursiva: Un usuario puede crear otros usuarios

2. Client (Cliente)

- Extiende información de usuarios con rol CLIENTE
- Información personal y de contacto
- Estados: ACTIVO, INACTIVO, SUSPENDIDO
- Propietarios de mascotas

3. Pet (Mascota)

- Pacientes del sistema veterinario
- Pertenece a un cliente (ManyToOne)
- Tiene una raza específica (ManyToOne)
- Información: nombre, fecha de nacimiento, sexo

4. Specie (Especie)

- Clasificación principal (Perro, Gato, etc.)
- Tiene múltiples razas (OneToMany)

5. Race (Raza)

- Clasificación específica de la especie
- Pertenece a una especie (ManyToOne)

Entidades de Gestión de Citas

6. Appointment (Cita)

- Programación de visitas veterinarias
- Relaciones:
 - Cliente (ManyToOne)
 - Mascota (ManyToOne)
 - Veterinario/Usuario (ManyToOne)
 - Sucursal (ManyToOne)
- Estados: PENDIENTE, CONFIRMADA, CANCELADA, ATENDIDA, NO_ASISTIO

7. Subsidiary (Sucursal)

- Locaciones de la clínica veterinaria
- Tiene horarios de atención (OneToMany)
- Administrada por un usuario (ManyToOne)

8. SubsidiarySchedule (Horario de Sucursal)

- Define horarios de atención por día y turno
- Pertenece a una sucursal (ManyToOne)

Entidades Médicas

9. MedicalHistory (Historial Clínico)

- Un historial por mascota
- Contiene múltiples consultas (OneToMany)
- Estados: ACTIVO, CERRADO

10. Consultation (Consulta)

- Registro de visita médica
- Pertenece a un historial clínico (ManyToOne)

- Atendida por un veterinario (ManyToOne)
- Contiene tratamientos (OneToMany)
- Información: motivo, diagnóstico, indicaciones

11. Treatment (Tratamiento)

- Tratamientos prescritos en una consulta
- Pertenece a una consulta (ManyToOne)
- Relacionado con una mascota (ManyToOne)
- Duración y observaciones

Entidades de Vacunación

12. Vaccine (Vacuna)

- Catálogo de vacunas disponibles
- Específica para una especie (ManyToOne)

13. VaccinationScheme (Esquema de Vacunación)

- Define el plan de vacunación
- Pertenece a una vacuna (ManyToOne)
- Especifica: número de dosis, edad recomendada
- Constraint único: vacuna + número de dosis

14. PetScheme (Esquema de Mascota)

- Asignación de esquema de vacunación a mascota
- Relación con mascota (ManyToOne)
- Relación con esquema de vacunación (ManyToOne)
- Estados: PENDIENTE, EN_PROCESO, COMPLETADO
- Constraint único: mascota + esquema

15. PetVaccination (Vacunación de Mascota)

- Registro de vacunación aplicada
- Relacionada con esquema de mascota (ManyToOne)
- Fecha de aplicación y próxima dosis

Entidades de Comunicación

16. Notification (Notificación)

- Sistema de notificaciones
- Puede estar relacionada con:
 - Cliente (ManyToOne)
 - Veterinario (ManyToOne)
 - Cita (ManyToOne)
 - Vacuna (ManyToOne)
- Tipos: CITA, VACUNA
- Estados: leída, enviada por email

Enumeraciones del Sistema

UserRol

- ROLE_VETERINARIO: Profesional veterinario
- ROLE_ADMINISTRADOR: Administrador del sistema
- ROLE_CLIENTE: Cliente/dueño de mascotas
- ROLE_ADMINISTRADOR_VETERINARIA: Administrador de sucursal

UserState

- Estados de activación de usuarios

ClientState

- ACTIVO: Cliente activo
- INACTIVO: Cliente inactivo
- SUSPENDIDO: Cliente suspendido

EstadoCita

- PENDIENTE: Cita programada pero no confirmada
- CONFIRMADA: Cita confirmada por el cliente
- CANCELADA: Cita cancelada
- ATENDIDA: Cita completada
- NO_ASISTIO: Cliente no asistió a la cita

MedicalHistoryState

- ACTIVO: Historial activo
- CERRADO: Historial cerrado

PetSchemeState

- PENDIENTE: Esquema asignado pero no iniciado
- EN_PROCESO: Vacunación en curso
- COMPLETADO: Esquema de vacunación completado

Sex

- MACHO
- HEMBRA

SubsidiaryState

- Estados de sucursales

ShiftType

- Tipos de turnos para horarios de sucursales

DayOfWeek

- Días de la semana para horarios

NotificationTitle

- Títulos predefinidos para notificaciones

CONFIGURACIÓN DEL SISTEMA

Archivo application.properties

El sistema utiliza variables de entorno para la configuración, permitiendo diferentes configuraciones por ambiente:

Configuración de Base de Datos

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

Descripción:

- DB_URL : URL de conexión a PostgreSQL (ej: jdbc:postgresql://localhost:5432/huellitasoft)
- DB_USERNAME : Usuario de base de datos
- DB_PASSWORD : Contraseña de base de datos
- ddl-auto=validate : Valida el esquema sin modificarlo automáticamente
- show-sql=true : Muestra las queries SQL en consola para debugging

Configuración de Seguridad OAuth2

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=${AUTH0_ISSUER_URI}
auth0.audience=${AUTH0_AUDIENCE}
auth0.roles=${AUTH0_ROLE}
app.post-reg.secret=${POST_REG_SECRET}
```

Descripción:

- AUTH0_ISSUER_URI : URI del emisor de tokens JWT (Auth0)
- AUTH0_AUDIENCE : Audiencia esperada en los tokens JWT
- AUTH0_ROLE : Namespace de roles en Auth0
- POST_REG_SECRET : Secreto para registro post-autenticación

Configuración CORS

```
web.cors.allowed-origins=${CORS_ALLOWED_ORIGINS}
```

Descripción:

- Lista separada por comas de orígenes permitidos para CORS
- Ejemplo: <http://localhost:5173>,<https://app.huellitasoft.com>

Configuración de Correo Electrónico

```
# Variables de entorno requeridas
MAIL_HOST=smtplib.org (u otro proveedor)
MAIL_PORT=587
MAIL_USERNAME=correo@ejemplo.com
MAIL_PASSWORD=contraseña_aplicacion
```

Configuración de Debugging y Logging

```
spring.security.debug=${WEB_SECURITY_DEBUG:false}
logging.level.org.springframework.security=${SPRING_SECURITY_LOG_LEVEL:INFO}
```

Configuración de Swagger

```
springdoc.swagger-ui.download-url=/v3/api-docs.yaml
```

Clases de Configuración

1. SecurityConfig.java

Propósito: Configuración central de seguridad del sistema.

Características principales:

- Autenticación OAuth2 con Auth0
- Autorización basada en roles
- Sesiones stateless (JWT)
- CSRF deshabilitado (apropiado para API REST)
- Configuración de endpoints públicos y protegidos

Reglas de autorización implementadas:

Endpoints públicos (sin autenticación):

- POST /api/users/register-from-auth0 : Registro de usuarios desde Auth0
- /swagger-ui/** , /v3/api-docs/** : Documentación de API
- /actuator/** : Endpoints de monitoreo

Endpoints por rol:

RAZAS:

- GET: Todos los usuarios autenticados
- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

ESPECIES:

- GET: Todos los usuarios autenticados
- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

MASCOTAS:

- GET: Todos los usuarios autenticados
- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA, CLIENTE

CLIENTES:

- GET: Todos los usuarios autenticados
- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

HISTORIALES CLÍNICOS:

- GET: Todos los usuarios autenticados
- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

CONSULTAS:

- GET: Todos los usuarios autenticados
- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

TRATAMIENTOS:

- GET: Todos los usuarios autenticados

- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

CITAS:

- GET: Todos los usuarios autenticados
- POST: Todos los usuarios autenticados
- PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

VACUNAS, ESQUEMAS DE VACUNACIÓN, ESQUEMAS DE MASCOTA, VACUNACIÓN DE MASCOTA:

- GET: Todos los usuarios autenticados
- POST, PUT, DELETE: VETERINARIO, ADMINISTRADOR, ADMINISTRADOR_VETERINARIA

2. CorsConfig.java

Propósito: Configuración de CORS (Cross-Origin Resource Sharing).

Características:

- Orígenes permitidos configurables mediante variables de entorno
- Soporte para patrones de origen (wildcards)
- Credenciales permitidas (cookies, headers de autorización)
- Métodos HTTP permitidos: GET, POST, PUT, PATCH, DELETE, OPTIONS
- Headers expuestos: Authorization, Location
- Preflight cache: 3600 segundos

3. CustomJwtAuthenticationConverter.java

Propósito: Conversión de tokens JWT a objeto de autenticación de Spring Security.

Funcionalidad:

- Extrae roles del claim de permisos en el JWT
- Convierte roles de Auth0 a GrantedAuthorities de Spring
- Maneja el namespace de roles configurado

4. AudienceValidator.java

Propósito: Validación de la audiencia del token JWT.

Funcionalidad:

- Verifica que el token JWT contenga la audiencia esperada

- Rechaza tokens con audiencia no válida

5. SwaggerConfig.java

Propósito: Configuración de documentación OpenAPI/Swagger.

Características:

- Información del proyecto: título, versión, descripción
- Esquema de seguridad Bearer JWT
- Aplicación global del esquema de seguridad
- Información de contacto y licencia

Acceso a la documentación:

- Swagger UI: <http://localhost:8080/swagger-ui.html>
- OpenAPI JSON: <http://localhost:8080/v3/api-docs>
- OpenAPI YAML: <http://localhost:8080/v3/api-docs.yaml>

6. MailConfig.java

Propósito: Configuración del servicio de correo electrónico.

Configuración SMTP:

- Host configurable (Gmail, SendGrid, etc.)
- Puerto SMTP
- Autenticación de usuario
- TLS/STARTTLS habilitado
- Debug mode configurable

7. WebSocketConfig.java

Propósito: Configuración de WebSocket para comunicación en tiempo real.

Configuración:

- Broker simple habilitado
- Destinos: `/topic` , `/queue`
- Prefijo de aplicación: `/app`
- Endpoint STOMP: `/ws`
- SockJS habilitado para fallback
- Origen permitido: <http://localhost:5173>

CAPA DE PERSISTENCIA

Repositorios JPA

Todos los repositorios extienden `JpaRepository<Entity, ID>` proporcionando operaciones CRUD básicas:

1. **AppointmentRepository**: Gestión de citas
2. **ClientRepository**: Gestión de clientes
3. **ConsultationRepository**: Gestión de consultas médicas
4. **MedicalHistoryRepository**: Gestión de historiales clínicos
5. **NotificationRepository**: Gestión de notificaciones
6. **PetRepository**: Gestión de mascotas
7. **PetSchemeRepository**: Gestión de esquemas de vacunación de mascotas
8. **PetVaccinationRepository**: Gestión de registros de vacunación
9. **RaceRepository**: Gestión de razas
10. **SpecieRepository**: Gestión de especies
11. **SubsidiaryRepository**: Gestión de sucursales
12. **SubsidiaryScheduleRepository**: Gestión de horarios de sucursales
13. **TreatmentRepository**: Gestión de tratamientos
14. **UserRepository**: Gestión de usuarios
15. **VaccinationSchemeRepository**: Gestión de esquemas de vacunación
16. **VaccineRepository**: Gestión de vacunas

Características de los Repositorios

Operaciones heredadas de `JpaRepository`:

- `save(Entity)` : Crear o actualizar entidad
- `findById(ID)` : Buscar por ID
- `findAll()` : Obtener todas las entidades
- `deleteById(ID)` : Eliminar por ID
- `count()` : Contar registros
- `existsById(ID)` : Verificar existencia

Estrategias de carga:

- `FetchType.LAZY` : Carga perezosa para relaciones (por defecto en `@ManyToOne`, `@OneToMany`)

- `FetchType.EAGER` : Carga inmediata cuando sea necesario

Cascadas:

- `CascadeType.ALL` : Operaciones en cascada para relaciones padre-hijo
- `orphanRemoval=true` : Eliminación de entidades huérfanas

MANEJO DE EXCEPCIONES

GlobalExceptionHandler

Manejador centralizado de excepciones que proporciona respuestas consistentes y estructuradas.

Excepciones Manejadas

1. ResourceNotFoundException

- HTTP Status: 404 NOT FOUND
- Uso: Cuando un recurso solicitado no existe
- Respuesta: `ErrorResponse` con detalles del error

2. ResourceAlreadyExistsException

- HTTP Status: 409 CONFLICT
- Uso: Cuando se intenta crear un recurso que ya existe
- Respuesta: `ErrorResponse` con detalles del conflicto

3. MethodArgumentNotValidException

- HTTP Status: 400 BAD REQUEST
- Uso: Errores de validación de datos de entrada
- Respuesta: Mapa de errores por campo

4. DataIntegrityViolationException

- HTTP Status: 409 CONFLICT
- Uso: Violaciones de integridad de base de datos
- Respuesta: Mensaje de error descriptivo

5. IllegalArgumentException

- HTTP Status: 400 BAD REQUEST
- Uso: Argumentos inválidos en operaciones
- Respuesta: Mensaje de error

6. Exception (genérica)

- HTTP Status: 500 INTERNAL SERVER ERROR
- Uso: Errores no controlados
- Respuesta: Mensaje genérico de error

Estructura de ErrorResponse

```
{  
  "status": 404,  
  "message": "Recurso no encontrado",  
  "timestamp": "2025-11-05 10:30:45",  
  "path": "/api/endpoint"  
}
```

SEGURIDAD Y AUTENTICACIÓN

Flujo de Autenticación

1. **Cliente solicita token a Auth0**
 - Usuario se autentica en Auth0
 - Auth0 emite un token JWT
2. **Cliente incluye token en peticiones**
 - Header: Authorization: Bearer <token>
3. **Backend valida token**
 - Verifica firma del token
 - Valida emisor (issuer)
 - Valida audiencia
 - Valida expiración
4. **Backend extrae roles**
 - CustomJwtAuthenticationConverter extrae roles
 - Convierte a GrantedAuthorities
5. **Spring Security autoriza**

- Verifica permisos según rol
- Permite o deniega acceso

Configuración de Auth0

Variables requeridas:

- AUTH0_ISSUER_URI : <https://your-tenant.auth0.com/>
- AUTH0_AUDIENCE : Identificador de la API en Auth0
- AUTH0_ROLE : Namespace de roles (ej: <https://huellitasoft.com/roles>)

Encriptación de Contraseñas

Se utiliza BCryptPasswordEncoder para el cifrado de contraseñas locales:

- Algoritmo: BCrypt
- Costo: 10 (por defecto)
- Salting automático

SERVICIOS COMPLEMENTARIOS

EmailService

Propósito: Envío de correos electrónicos del sistema.

Funcionalidades:

- Envío de correos transaccionales
- Notificaciones de citas
- Recordatorios de vacunación
- Soporte para plantillas Thymeleaf

NotificationService

Propósito: Gestión del sistema de notificaciones.

Funcionalidades:

- Creación de notificaciones

- Marcado de leídas/no leídas
- Envío por email
- Notificaciones por WebSocket

WebSocketService

Propósito: Comunicación en tiempo real.

Funcionalidades:

- Notificaciones push a clientes conectados
- Actualizaciones en tiempo real de citas
- Alertas inmediatas

Endpoints WebSocket:

- Conexión: /ws
- Topics: /topic/notifications , /queue/user-specific

CONSTRUCCIÓN Y DESPLIEGUE

Construcción con Maven

Compilación:

```
mvn clean compile
```

Empaquetado:

```
mvn clean package
```

Ejecución:

```
mvn spring-boot:run
```

Generación de JAR:

```
mvn clean package -DskipTests
```

El JAR ejecutable se genera en: `target/huellitasoft-0.0.1-SNAPSHOT.jar`

Plugins de Maven Configurados

1. maven-compiler-plugin

- Procesamiento de anotaciones de Lombok
- Compatibilidad con Java 21

2. spring-boot-maven-plugin

- Empaquetado de aplicación Spring Boot
- Exclusión de Lombok del JAR final

Ejecución del JAR

```
java -jar target/huellitasoft-0.0.1-SNAPSHOT.jar
```

Variables de Entorno Requeridas

Antes de ejecutar la aplicación, configurar:

```
# Base de datos
export DB_URL=jdbc:postgresql://localhost:5432/huellitasoft
export DB_USERNAME=usuario
export DB_PASSWORD=password

# Auth0
export AUTH0_ISSUER_URI=https://tu-tenant.auth0.com/
export AUTH0_AUDIENCE=https://api.huellitasoft.com
export AUTH0_ROLE=https://huellitasoft.com/roles

# CORS
export CORS_ALLOWED_ORIGINS=http://localhost:5173

# Email
export MAIL_HOST=smtp.gmail.com
export MAIL_PORT=587
export MAIL_USERNAME=tu-email@gmail.com
export MAIL_PASSWORD=tu-password-de-aplicacion

# Otros
export POST_REG_SECRET=tu-secreto
```

O utilizar un archivo `.env` con `spring-dotenv`.

TESTING

Estructura de Tests

```
src/test/java/huellitasoft_web/huellitasoft/
└─ HuellitasoftApplicationTests.java
```

Dependencias de Testing

- **spring-boot-starter-test:** Framework de testing de Spring Boot
- **spring-security-test:** Utilities para testing de seguridad
- **JUnit 5:** Framework de pruebas unitarias
- **Mockito:** Framework de mocking

- **AssertJ:** Biblioteca de aserciones

Ejecución de Tests

```
mvn test
```

LOGGING Y MONITOREO

Configuración de Logging

Framework: SLF4J con Logback (incluido en Spring Boot)

Niveles configurables:

- `logging.level.org.springframework.security` : Logging de seguridad
- `spring.jpa.show-sql` : Muestra queries SQL

Niveles de logging:

- ERROR: Errores críticos
- WARN: Advertencias
- INFO: Información general (por defecto)
- DEBUG: Información detallada de debugging
- TRACE: Información muy detallada

Endpoints de Actuator

Spring Boot Actuator proporciona endpoints de monitoreo:

- `/actuator/health` : Estado de salud de la aplicación
- `/actuator/info` : Información de la aplicación
- `/actuator/metrics` : Métricas de la aplicación

MEJORES PRÁCTICAS IMPLEMENTADAS

Separación de Responsabilidades

- Controladores solo manejan HTTP
- Servicios contienen lógica de negocio
- Repositorios solo acceden a datos
- DTOs separan API de modelo de dominio

Validación de Datos

- Validación con Jakarta Bean Validation
- Anotaciones como @NotNull, @NotBlank, @Email, @Size
- Validación en DTOs de entrada

Manejo de Errores

- Excepciones personalizadas para casos específicos
- GlobalExceptionHandler centralizado
- Respuestas de error consistentes

Seguridad

- Autenticación stateless con JWT
- Autorización basada en roles
- CORS configurado apropiadamente
- Contraseñas encriptadas con BCrypt

Documentación

- Swagger/OpenAPI para documentación interactiva
- Anotaciones @Operation, @ApiResponse
- Documentación de esquemas de datos

Transaccionalidad

- Uso de @Transactional en servicios
- Gestión automática de transacciones
- Rollback en caso de errores

Uso de Lombok

- Reducción de código boilerplate
- `@Data`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`
- Mejora de legibilidad

Este documento técnico proporciona una visión completa de la arquitectura, configuración y componentes del sistema Huellitasoft Backend. Para información sobre casos de uso específicos y guías de implementación, consultar el documento de `DOCUMENTACION_FUNCIONAL.md`.